

Practica 2. Sintaxis

Algunos apuntes que tengo para realizar la practica 2:

$G = \{ N, T, S, P \}$

N = Simbolos no terminales => Expresiones

```
N = { <numero_entero>, <digito> }
```

ej :

T = Simbolos terminales

ej:

```
T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
```

S = Punto de partida, desde el cual se generan todas las expresiones.

```
S = <numero_entero>
```

P = Conjunto de producciones

```
<numero_entero> ::= <digito> | <digito> <numero_entero>  
<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Ejemplo:

```
G = ( N, T, S, P )  
N = { <numero_entero>, <digito> }  
T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }  
S = <numero_entero>  
P = {  
  <numero_entero> ::= <digito> <numero_entero> | <numero_entero> <digito> | <digito>  
  <digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  
}
```

Para la resolucion de ejercicios, no confundir BNF con EBNF.

- BNF tiene recursion
- EBNF **NO TIENE RECURSION**, es repetitivo.

Ejercicio 1

Meta símbolos utilizados por		Significado
BNF	EBNF	
< >	< >	Definición de un elemento no terminal
::=	::=	Definición de una producción
	()	Selección de una alternativa
< p > < p1 >	{ }	Repetición
	*	Repetición de 0 o más veces
	+	Repetición de 1 o más veces
	[]	Opcional está presente o no lo está
Nota: p y p1 son producciones si		

Ejercicio 2

La **sintaxis** es fundamental en un lenguaje de programación porque define las **reglas estructurales** que determinan cómo se pueden escribir expresiones, instrucciones y programas válidos. Si un código no cumple la sintaxis del lenguaje, el compilador o intérprete mostrará un error y no lo ejecutará.

◆ Elementos de la sintaxis:

1. **Alfabeto:** Conjunto de símbolos válidos (letras, números, operadores, etc.).
2. **Palabras reservadas:** Identificadores con un significado especial (`if` , `while` , `return` en C, Java, Python, etc.).
3. **Reglas gramaticales:** Cómo se combinan los símbolos para formar expresiones válidas.
4. **Estructura jerárquica:** Organización del código (bloques, indentación en Python, llaves `{ }` en C/Java).

Ejercicio 3

✅ Regla lexicográfica:

- Se refiere a la formación de **palabras válidas** en un lenguaje.
- Define cómo se escriben identificadores, números, operadores y palabras clave.
- Se describe mediante **expresiones regulares** o **autómatas finitos**.

✅ Regla sintáctica:

- Define la **estructura y organización** de las palabras en una oración válida.
- Se especifica mediante gramáticas formales como **BNF**.

✅ Regla sintáctica: El `if` debe ir seguido de un `( )` con una **expresión booleana** y un

bloque de código `{}` .

Ejercicio 4

✓ Palabras reservadas

Son identificadores con un **significado especial** en el lenguaje y no pueden usarse como nombres de variables, funciones o clases.

✓ Equivalente en una gramática

En **BNF**, una palabra reservada es un **símbolo terminal**, ya que no se expande en más reglas. En **Java**, `public` es una palabra reservada utilizada para definir el acceso a una clase o método:

Ejercicio 5

- a)

```
G=(N, T , S , P)
N= No terminales
T= Terminales
S= Simbolo inicial
P= Conjunto de producciones

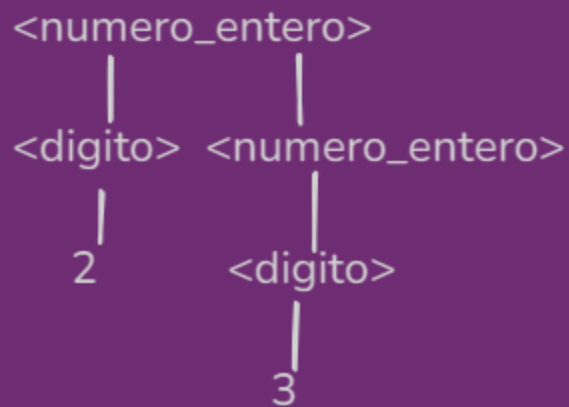
N = { <numero_entero> , <digito> }
T = { 0,1,2,3,4,5,6,7,8,9 }
S= <numero_entero>
P = {
<numero_entero>::=<digito><numero_entero> | <numero_entero><digito> | <digito>
<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
}
```

- b) Esta solución es ambigua porque una misma cadena puede ser derivada de múltiples maneras, es decir, tiene más de un árbol de derivación posible.

por ejemplo:

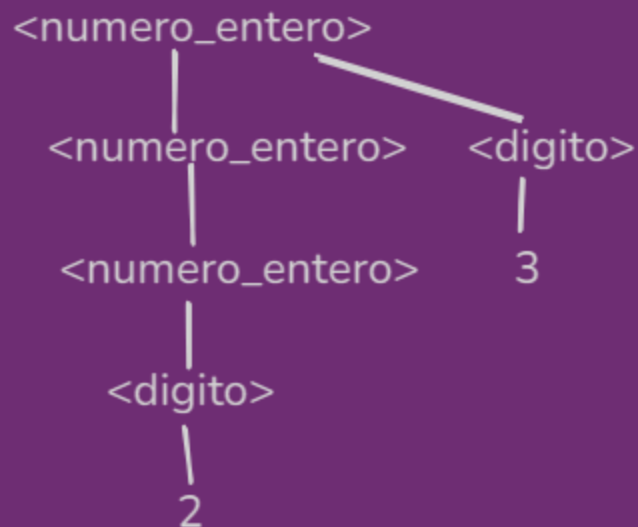
de izquierda a derecha

+23



de derecha a izquierda

+23



- Una vez dejado en claro que este conjunto de producciones permite realizar dos arboles de derivacion para la misma expresion, por lo tanto produce ambigüedad, lo podemos resolver Sacando la segunda opcion dentro de <numero_entero>, por lo tanto, nuestra gramatica quedaria:

$G=(N, T, S, P)$

N= No terminales

T= Terminales

S= Simbolo inicial

P= Conjunto de producciones

$N = \{ \langle \text{numero_entero} \rangle, \langle \text{digito} \rangle \}$

$T = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$

S= $\langle \text{numero_entero} \rangle$

P = {

$\langle \text{numero_entero} \rangle ::= \langle \text{digito} \rangle \langle \text{numero_entero} \rangle \mid \langle \text{digito} \rangle$

$\langle \text{digito} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

}

De esta manera, no generamos ambigüedad, y el árbol se deriva de izquierda a derecha, como suele ser normal leer los números.

Ejercicio 6

BNF para la definición de una palabra cualquiera.

BNF PARA UNA PALABRA CUALQUIERA

$G = (N, T, S, P)$

$N = \{ \langle \text{PALABRA} \rangle, \langle \text{CARACTER} \rangle \}$

$T = \{ "a" \dots "z", "A" \dots "Z", " " \}$ // ASUMO QUE LA PALABRA NO PUEDE TENER ESPACIOS

$S = \langle \text{PALABRA} \rangle$

$P = \{$

$\langle \text{PALABRA} \rangle := \langle \text{CARACTER} \rangle \langle \text{PALABRA} \rangle \mid \langle \text{CARACTER} \rangle$

$\langle \text{CARACTER} \rangle := "a" \mid "b" \mid \dots \mid "z" \mid "A" \mid "B" \mid \dots \mid "Z"$

$\}$

ÁRBOL DE DERIVACIÓN:

$E \Rightarrow$ GIANCA

$\langle \text{PALABRA} \rangle$

$\mid$
 $\langle \text{CARACTER} \rangle \langle \text{PALABRA} \rangle$

$\mid$
G

$\mid$
 $\langle \text{CARACTER} \rangle \langle \text{PALABRA} \rangle$

$\mid$
I

$\mid$
 $\langle \text{CARACTER} \rangle \langle \text{PALABRA} \rangle$

$\mid$
A

$\mid$
 $\langle \text{CARACTER} \rangle \langle \text{PALABRA} \rangle$

$\mid$
N

$\mid$
 $\langle \text{CARACTER} \rangle \langle \text{PALABRA} \rangle$

$\mid$
C

$\mid$
 $\langle \text{PALABRA} \rangle$

$\mid$
 $\langle \text{CARACTER} \rangle$

$\mid$
A

Ejercicio 7

Ejercicio 7 Parte 2 Definir GJBNF y EBNF (alternativa) para definir números reales

CON EBNF

$$G = (N, T, S, P)$$

$$N = \{ \langle \text{NUMERO_REAL} \rangle, \langle \text{DIGITO} \rangle \}$$

$$T = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., - \}$$

$$S = \langle \text{NUMERO_REAL} \rangle$$

$$P = \{$$

$$\langle \text{NUMERO_REAL} \rangle ::= [-] \{ \langle \text{DIGITO} \rangle \}^+ \cdot "." \{ \langle \text{DIGITO} \rangle \}^+ \}$$

$$\langle \text{DIGITO} \rangle ::= 0/1/2/3/4/5/6/7/8/9$$

$$\}$$

Coco que esta es la forma más fácil, sin tener que tener los cuantos no terminan

ADICION: Asumi que al ser un numero real, siempre va a tener parte decimal, por ejemplo si se quiere definir a numero 3 con esta sintaxis debería ser 3.0

CON BNF:

$$G = (N, T, S, P)$$

$$N = \{ \langle \text{EXPRESION_REAL} \rangle, \langle \text{NUMERO_REAL} \rangle, \langle \text{ENTERO} \rangle, \langle \text{DIGITO} \rangle, \langle \text{SEPARADOR_DECIMAL} \rangle, \langle \text{SIGNO_MENOS} \rangle \}$$

$$T = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., - \}$$

$$S = \{ \langle \text{EXPRESION_REAL} \rangle \}$$

$$P = \{$$

$$\langle \text{EXPRESION_REAL} \rangle ::= \langle \text{SIGNO_MENOS} \rangle \langle \text{NUMERO_REAL} \rangle / \langle \text{NUMERO_REAL} \rangle$$

$$\langle \text{NUMERO_REAL} \rangle ::= \langle \text{ENTERO} \rangle \langle \text{SEPARADOR_DECIMAL} \rangle \langle \text{ENTERO} \rangle$$

$$\langle \text{ENTERO} \rangle ::= \langle \text{DIGITO} \rangle \langle \text{ENTERO} \rangle / \langle \text{DIGITO} \rangle$$

$$\langle \text{DIGITO} \rangle ::= 0/1/2/3/4/5/6/7/8/9$$

$$\langle \text{SEPARADOR_DECIMAL} \rangle ::= "."$$

$$\langle \text{SIGNO_MENOS} \rangle ::= "-"$$

$$\}$$

Lo que cambia entre EBNF y BNF es que:

- En BNF Tenemos recursion.
- En EBNF No tenemos recursion. tenemos repeticion.

Ejercicio 9

EXERCÍCIO 9 PARTE 2

